MongoDB Atlas Vector Search

MongoDB Atlas is used as the vector database for storing and retrieving the embedded ROS messages.

Prerequisite: A MongoDB Atlas account is required to deploy and configure the vector database used by the system.

1. Create an Atlas Cluster Create a MongoDB Atlas cluster using the default configuration. The experiments were conducted using a **MongoDB Atlas Free cluster (formerly M0) running MongoDB 8.0.6**.

2. Create a Database User
Create a database user and configure the required username, password, and database access privileges.

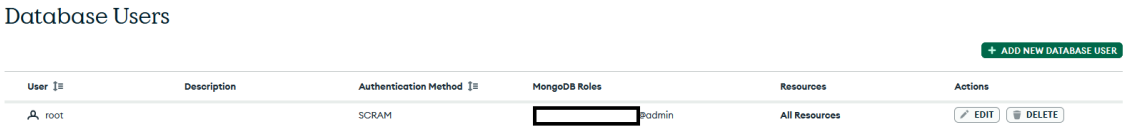


Figure 1. Create Database User.

3. Configure Network Access
From **Security → Database & Network Access → IP Access List → Add IP Address**
add the IP address of the machine from which the application will connect to MongoDB Atlas.

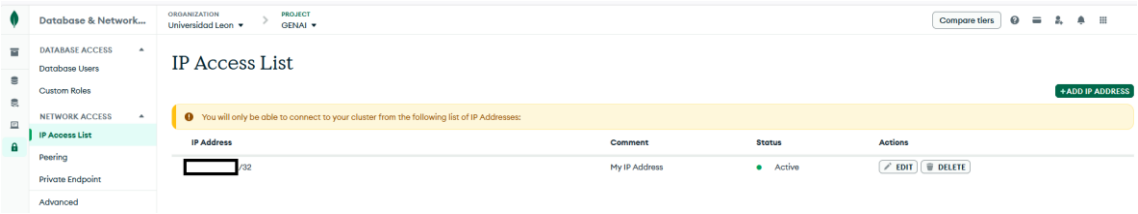


Figure 2. Configure Network Access.

4. Obtain the Connection String
In the Atlas cluster, select **Connect** and choose **Compass** to obtain the MongoDB connection string.

The connection string has the following format:

mongodb+srv://<username>:<db_password>@cluster0.XXXXXX.mongodb.net/

Replace <username> and <db_password> with the credentials of the database user created above.

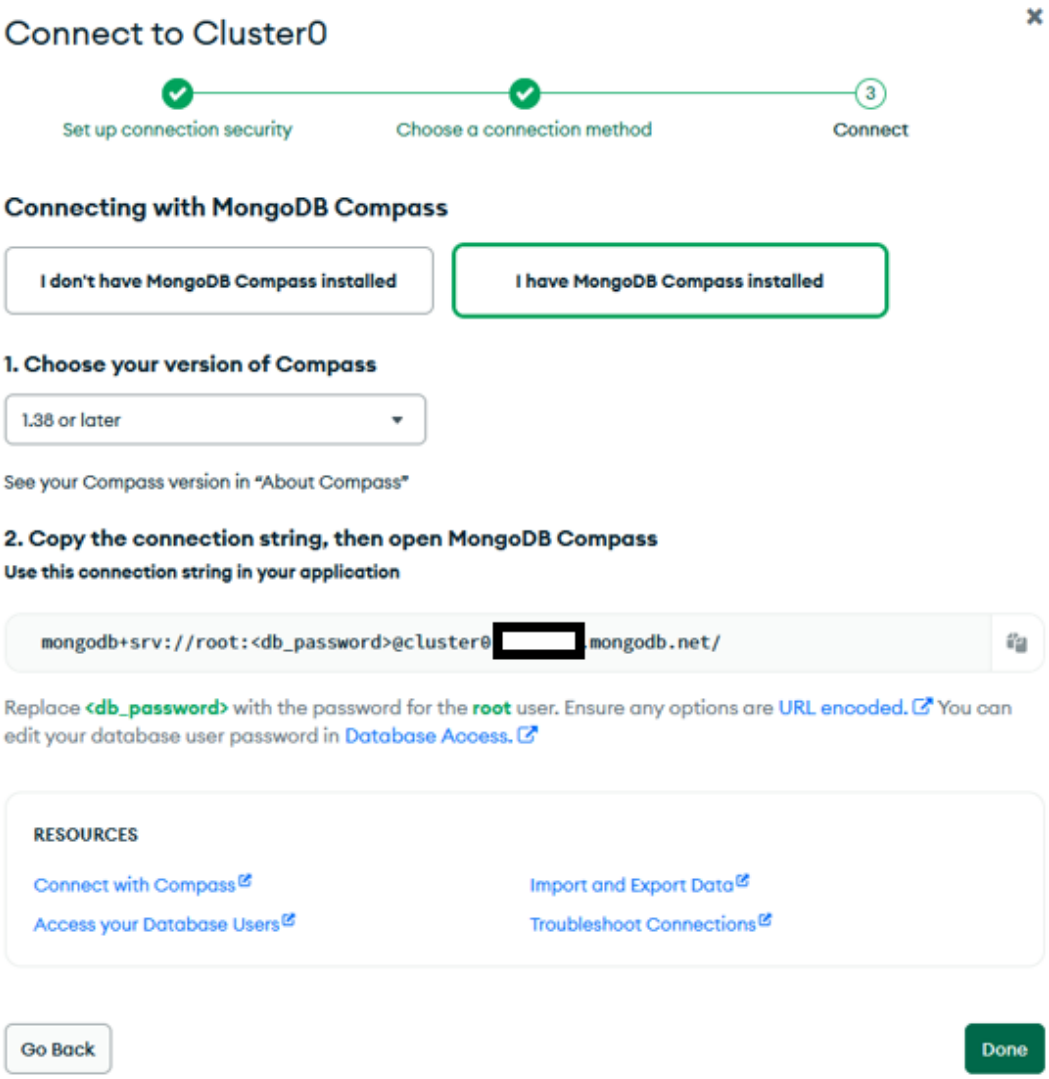


Figure 3. Obtain the MongoDB connection string.

5. Create the Database and Collection

Create the database and collection required by the application. The collection stores the ROS messages together with their corresponding vector embeddings.

From **Database → Clusters**, select your cluster and choose **Browse Collections.**
Specify the database and collection names.

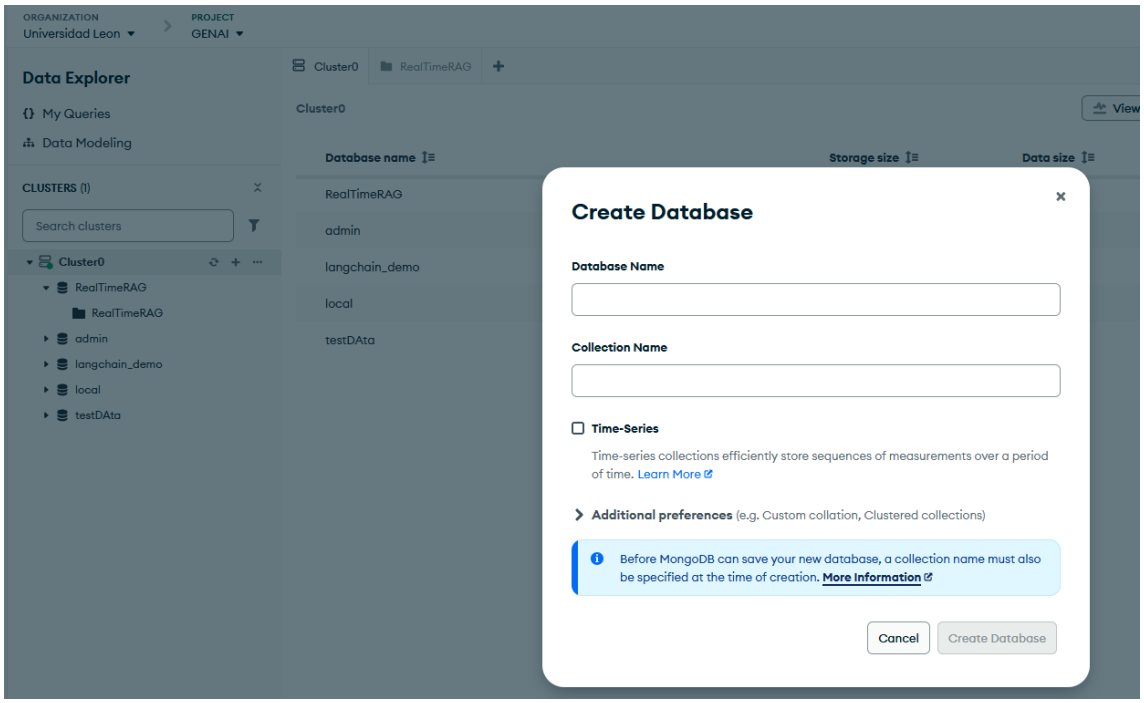


Figure 4. Create the database and collection.

6. Configure MongoDB Atlas Search and Vector Search

From **Database → Search & Vector Search**, click **CREATE SEARCH INDEX**

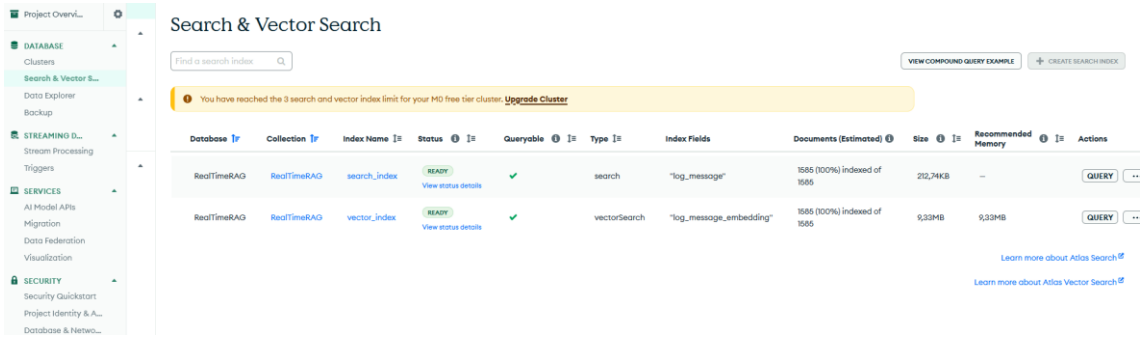


Figure 5. Accessing Search & Vector Search configuration in MongoDB Atlas.

The retrieval component uses a hybrid search approach that combines lexical and vector-based retrieval. MongoDB Atlas Search is used for lexical retrieval based on BM25, while MongoDB Atlas Vector Search is used to retrieve semantically similar documents based on their vector embeddings. The results from both retrieval methods are combined using Reciprocal Rank Fusion (RRF).

The fields indexed by both search mechanisms correspond to the document structure generated by the Kafka producer described in the previous section. In particular, `log_message` contains the ROS message text, while `log_message_embedding` contains its corresponding vector embedding.

Two search indexes must therefore be configured for the collection: one for lexical search and one for vector search.

6.1. Configure the Search Index

Create a **Search Index** named `search_index` for the collection using the following configuration:

```
{
  "mappings": {
    "dynamic": false,
    "fields": {
      "log_message": [
        {
          "type": "string"
        }
      ]
    }
  }
}
```

This index enables lexical retrieval of ROS messages using the log_message field and the BM25 ranking algorithm.

6.2. Configure the Vector Search Index

Create a Vector Search Index named vector_index for the collection using the following configuration:

```
{
  "fields": [
    {
      "numDimensions": 1536,
      "path": "log_message_embedding",
      "similarity": "dotProduct",
      "type": "vector"
    }
  ]
}
```

The numDimensions value must match the dimensionality of the embeddings generated by the embedding model used by the application. In this implementation, the embedding vectors have 1536 dimensions.

The retrieval component combines the results obtained from the lexical and vector searches using **Reciprocal Rank Fusion (RRF)**. The corresponding retrieval configuration is described in the application configuration section.

7. Configure API Keys

The Atlas Trigger uses external API services to generate vector embeddings.

The corresponding API keys must therefore be stored securely in MongoDB Atlas.

In the MongoDB Atlas Project, go to **STREAMING DATA → Triggers**, click **View All Apps**.

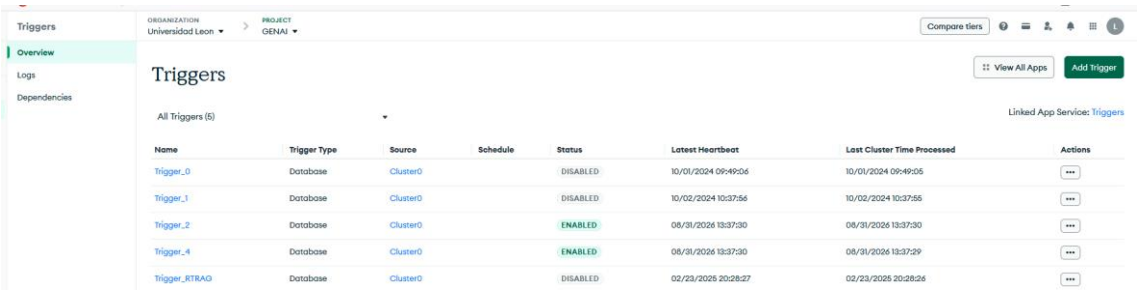


Figure 6. Accessing Triggers in MongoDB Atlas.

From the App Service interface, select Triggers→ Values.

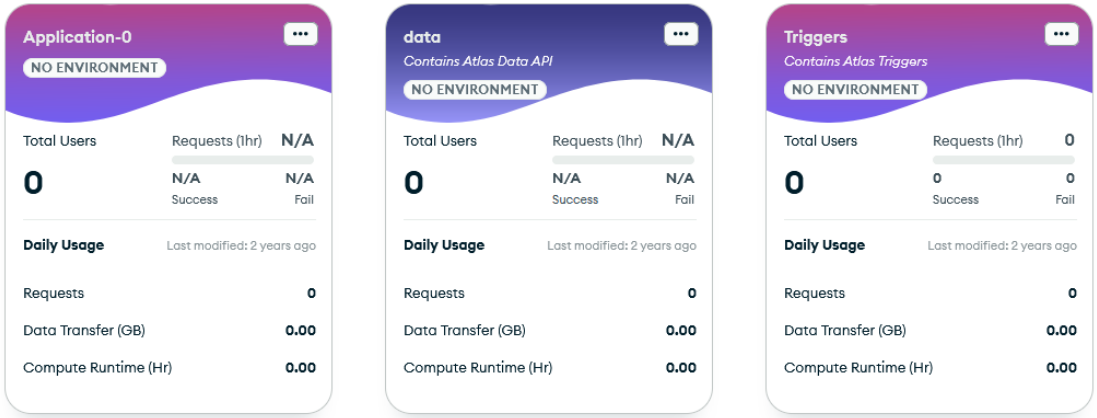


Figure 7. Accessing Trigger App Service.

7.1. Configure API Keys

Create a separate **Secret** for the API key required by the Trigger.

Click **Create a Value**, select **Secret** as the value type, provide a name for the secret, and enter the corresponding API key.

For example:
- OPENAI_API_KEY for the OpenAI API key.
- HUGGINGFACE_API_KEY for the Hugging Face API key.

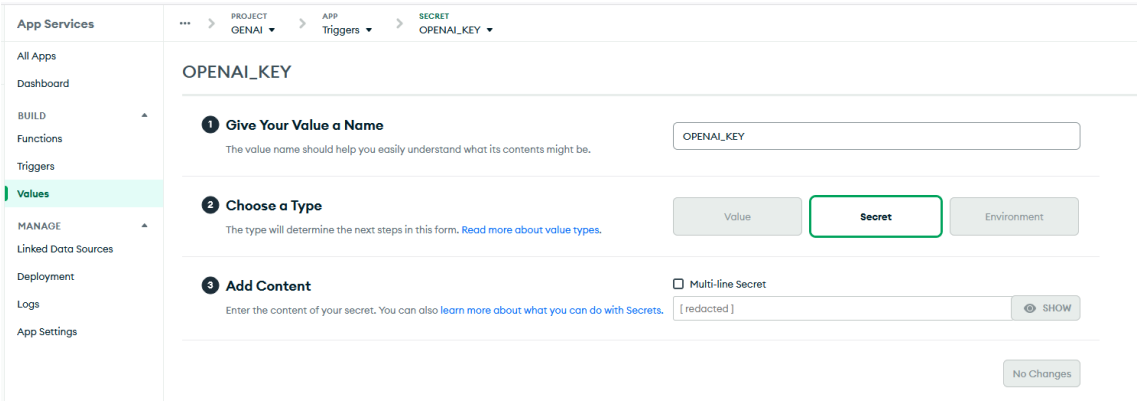


Figure 8. Creating a Secret to securely store an API key.

The API keys are stored as Secrets and are not directly accessible after they are saved. This prevents sensitive credentials from being exposed in the Trigger configuration.

7.2. Create Values Linked to the Secrets

For each Secret, create a corresponding **Value** that links to the Secret.

Click **Create a Value**, provide a name for the Value, select **Link to Secret**, and select the corresponding Secret.

Create one Value for each API key required by the Trigger.

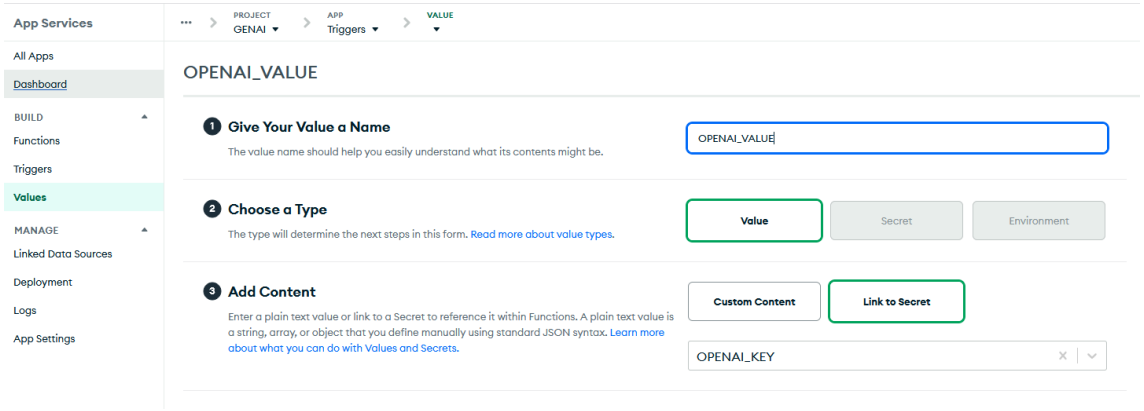


Figure 9. Creating a Value linked to a Secret.

8. Create the MongoDB Atlas Trigger

MongoDB Atlas Triggers are used to automatically generate vector embeddings when new ROS 2 messages are inserted into the collection.

A Trigger can be configured with a Function that calls an external embedding service through its API. The implementation supports two embedding alternatives: **OpenAI** and **Hugging Face**.

The Trigger must be configured to execute when a new document is inserted into the target collection.

8.1. Configure the Trigger

From the MongoDB Atlas project, go to Streaming Data → Triggers and select Add Trigger. Configure the Trigger as follows:

- Trigger Type: Database
- Watch Against: Collection
- Cluster Name: <CLUSTER_NAME>
- Database: <DATABASE_NAME>
- Collection: <COLLECTION_NAME>
- Operation Type: Insert Document
- Full Document: Enabled
- Event Ordering: Enabled

The Trigger must be configured to execute the corresponding Function when a new document is inserted into the collection.

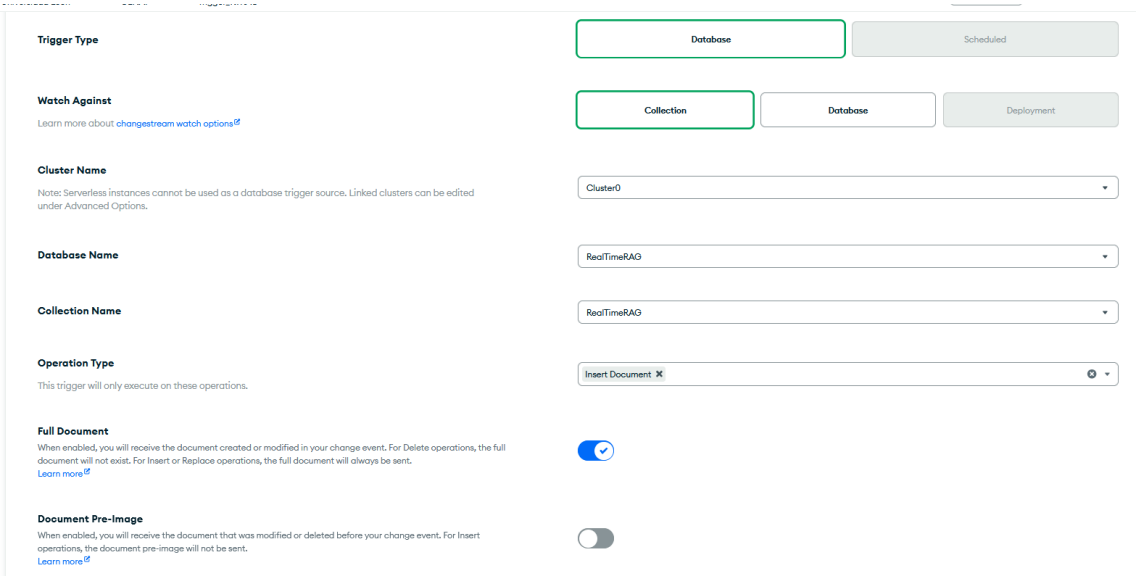


Figure 10. MongoDB Atlas Trigger configuration.

8.2. Configure the OpenAI Embedding Function

The OpenAI implementation uses the OpenAI Embeddings API with the text-embedding-ada-002 model, which was used in the experiments. Other compatible embedding models can also be used. When using a different model, the corresponding embedding dimensionality must be specified in the MongoDB Atlas Vector Search index configuration.

The Function retrieves the OpenAI API key from the Value named OPENAI_VALUE and sends the contents of the log_message field to the OpenAI Embeddings API. The generated embedding is then stored in the same MongoDB document in the log_message_embedding field.

Replace <SERVICE_NAME>, <DATABASE_NAME>, and <COLLECTION_NAME> in the Function with the names of the MongoDB Atlas cluster, database, and collection configured in the previous steps.

The Function used for the OpenAI-based embedding generation is:

```
#### openAI trigger ######
exports = async function(changeEvent) {
    // Get the full document from the change event.
    const doc = changeEvent.fullDocument;
    // Define the OpenAI API url and key.
    const url = 'https://api.openai.com/v1/embeddings';
    // Use the name you gave the value of your API key in the "Values"
utility inside of App Services
    const openai_key = context.values.get("OPENAI_VALUE");
    try {
        console.log(`Processing document with id: ${doc._id}`);

        // Call OpenAI API to get the embeddings.
        let response = await context.http.post({
            url: url,
             headers: {
                'Authorization': [`Bearer ${openai_key}`],
                'Content-Type': ['application/json']
            },
```

```
            body: JSON.stringify({
                // Generate an embedding from the ROS message in
log_message
                input: doc.log_message,
                model: "text-embedding-ada-002"
            })
        });

        // Parse the JSON response
        let responseData = EJSON.parse(response.body.text());

        // Check the response status.
        if(response.statusCode === 200) {
            console.log("Successfully received embedding.");

            const embedding = responseData.data[0].embedding;
            console.log(embedding.length);

            // Use the name of your MongoDB Atlas Cluster
            const collection =
context.services.get("<SERVICE_NAME>").db("<DATABASE_NAME>").collection("
<COLLECTION_NAME>");

            // Update the document in MongoDB.
            const result = await collection.updateOne(
                { _id: doc._id },
                // The name of the new field you'd like to contain
your embeddings.
                { $set: { log_message_embedding: embedding }}
            );

            if(result.modifiedCount === 1) {
                console.log("Successfully updated the document.");
            } else {
                console.log("Failed to update the document.");
            }
        } else {
            console.log(`Failed to receive embedding. Status code:
${response.statusCode}`);
        }

    } catch(err) {
        console.error(err);
    }
};
```

8.3. Configure the Hugging Face Embedding Function

The Hugging Face implementation provides an alternative to the OpenAI-based embedding generation for users who do not want to use the OpenAI API. The provided configuration uses the intfloat/multilingual-e5-large. When using intfloat/multilingual-e5-large, the Vector Search index must be configured with numDimensions: 1024. Other compatible Hugging Face embedding models can also be used, provided that the corresponding embedding dimensionality and MongoDB Atlas Vector Search configuration are updated accordingly.

The Function retrieves the Hugging Face API key from the Value named HUGGINGFACE_VALUE and sends the contents of the log_message field to the Hugging Face API. The resulting embedding is stored in the log_message_embedding field.

Replace <SERVICE_NAME>, <DATABASE_NAME>, and <COLLECTION_NAME> in the Function with the names of the MongoDB Atlas cluster, database, and collection configured in the previous steps.

```
exports = async function(changeEvent) {
    // Get the full document from the change event.
    const doc = changeEvent.fullDocument;
    // Define the Hugging Face API url and key.
    const url = 'https://api-
inference.huggingface.co/models/intfloat/multilingual-e5-large';
    // Use the name you gave the value of your API key in the "Values"
utility inside of App Services
    const hf_key = context.values.get("HUGGINGFACE_VALUE");
    try {
        console.log(`Processing document with id: ${doc._id}`);

        // Call Hugging Face API to get the embeddings.
        let response = await context.http.post({
            url: url,
            headers: {
                'Authorization': [`Bearer ${hf_key}`],
                'Content-Type': ['application/json'],
                'x-wait-for-model': ['true']
            },
            //body: JSON.stringify({ inputs: doc.log_message }),
            body: JSON.stringify(doc.log_message),
            encodeBodyAsJSON: true
        });

        // Parse the JSON response
        let responseData = EJSON.parse(response.body.text());
        console.log(`Response: ${JSON.stringify(responseData)}`);

        // Check the response status.
        if(response.statusCode === 200) {
           console.log("Successfully received embedding.");

           const embedding = responseData;
           //const embedding = responseData[0].embedding;
           console.log(embedding.length);

            // Use the name of your MongoDB Atlas Cluster
            const collection =
context.services.get("<SERVICE_NAME>").db("<DATABASE_NAME>").collection("
<COLLECTION_NAME>");

            // Update the document in MongoDB.
            const result = await collection.updateOne(
                { _id: doc._id },
                // The name of the new field you'd like to contain
your embeddings.
                { $set: { log_message_embedding: embedding }}
            );

            if(result.modifiedCount === 1) {
                console.log("Successfully updated the document.");
            } else {
                console.log("Failed to update the document.");
            }
```

```
        } else {
            console.log(`Failed to receive embedding. Status code:
${response.statusCode}`);
        }

    } catch(err) {
        console.error(err);
    }
};
```

The OpenAI and Hugging Face implementations provide alternative approaches for generating embeddings. Only the Function corresponding to the embedding model used by the application should be associated with the Trigger.

The embedding field generated by the selected Function must correspond to the field configured in the MongoDB Atlas Vector Search index.